

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

“C. Shannon transformó ntra manera d entender la info al demostrar q en el caos y la incertidumbre existen patrones q podemos usar p/comprimir y optimizar los datos. Su teoría revolucionó telecoms y mostró q cn correcta interpretación, es posible reducir la complejidad sin perder el mensaje esencial. La compresión es el arte d decir + cn -, y Shannon mostró q este principio es el corazón d la eficiencia n el mundo digital”

ÁREA TEMÁTICA No1

PRÁCTICO 3 TEORÍA DE LA INFORMACIÓN: “Compactadores”

1. Sean los siguientes códigos:

s	p_i	a	b	c	d	e	f
s_1	1/2	000	0	0	0	0	0
s_2	1/4	001	01	10	10	10	100
s_3	1/16	010	011	110	110	1100	101
s_4	1/16	011	0111	1110	1110	1101	110
s_5	1/16	100	01111	11110	1011	1110	111
s_6	1/16	101	011111	111110	1101	1111	001

- Clasificar cada uno de ellos (Bloque, No singular, Instantáneos, Unívocamente decodif.)
 - Calcular la longitud promedio de los unívocamente decodificables.
 - En los códigos que presenten problemas, indicar qué falla y proponer una modificación mínima.
2. Ejercicios
- ¿Cuál de los conjuntos de longitudes de la siguiente tabla es válida para un código unívocamente decodificable con base $X\{0,1,2\}$?
 - Construir un código instantáneo con cada conjunto de longitudes válidas

Long. palabra →	1	2	3	4	5
Código A	2	1	2	1	4
Código B	2	2	2	3	1
Código C	1	4	6	0	0
Código D	2	2	2	2	3

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

3. Demostrar que el código coma cumple Kraft
4. Sea un código trinario. Codificar los diez dígitos decimales con longitudes 1, 2, 2, 2, 2, 2, 3, 3, 3, 3. Si no se pudiera ¿hasta cuantos dígitos podría codificar?, Realice el árbol de codificación.
5. Entropía y codificación estadística: Sea la prob. de la fuente: $A=0.30$, $B=0.25$, $C=0.18$, $D=0.12$, $E=0.08$, $F=0.07$
 - a. Calcular la entropía $H(x)$
 - b. Construir los códigos de Shannon, Fano y Huffman
 - c. Calcular para cada método la longitud promedio L , la eficiencia $\eta = H(X)/L$ y la redundancia $R=1-\eta$
 - d. Comparar los resultados y justificar cuál se aproxima mejor al límite teórico.
6. Método de diccionario: LZ y LZW. Trabajar con la cadena:
ERRECONERRECIGARROERRECONERREBARRIL
 - a. Aplicar LZ utilizando una ventana de búsqueda de 6 símbolos y una ventana de anticipación de 4 símbolos. Mostrar las tuplas generadas y reconstruir la cadena
 - b. Aplicar **LZW** usando como diccionario inicial los símbolos distintos de la cadena. Mostrar la evolución del diccionario, la secuencia de códigos emitidos y la descompresión.
 - c. Comparar qué tipo de repetición aprovecha cada método.
7. Codificación aritmética: Sea la fuente con $p(A)=0,50$; $p(B)=0,30$; $p(C)=0,20$ y la secuencia: ABACA
 - a. Codificar la secuencia mediante codificación aritmética. Mostrar los intervalos iniciales, las subdivisiones sucesivas y determinar el intervalo final. Elegir un valor perteneciente a dicho intervalo como representación de la secuencia.
 - b. A partir del valor obtenido, realizar el proceso de decodificación y verificar que se recupera la secuencia original.

FUENTES DE MARKOV

1. Dado el siguiente enunciado construir un conjunto de probabilidades condicionadas (con cuatro estados) en un proceso de Markov de primer orden, encontrar la matriz de transición y el grafo.

AABABACABABACABBABAACAB|AC

2. A partir del ejercicio nº1
 - a. Obtener la matriz estacionaria (distribución a la que tiende).
 - b. Comprobar que las potencias sucesivas de la matriz de transición tienen menor variabilidad en los valores de sus componentes.
 - c. Comprobar que la distribución estacionaria es la misma para el cuadrado de la matriz de transición.
 - d. ¿Es un proceso ergódico?
3. Demostrar que las filas de la matriz de transición son linealmente dependientes en un proceso de Markov.
4. Analizar el siguiente ejemplo y responder los enunciados:
Sea $S = (s_1, s_2, s_3, s_4)$ $p(s_i/s_{j1}, s_{j2}, s_{j3})$
 - a. ¿De qué orden es el sistema de Markov?.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- b. ¿En cuántos estados puede encontrarse el sistema?
 - c. Supuestas conocidas todas las probabilidades condicionales ¿cuál será la forma del sistema de ecuaciones para encontrar las $p(s_i)$?
5. Supongamos una máquina con dos estados, a y b , $a=1$, $b=2$. Estudiamos el proceso día a día llegando a comprobar que:
 - i. si a hoy, hay un 75% de chances que a mañana
 - ii. si no a hoy, la arreglan mañana
- a. Encontrar la matriz de transición
 - b. Calcular hasta la quinta potencia de la matriz de transición y analizar los resultados
 - c. Encontrar la matriz de equilibrio o estacionaria
6. Dado un proceso de Markov con tres estados a, b, c ; con las siguientes probabilidades: $p(a/a)=3/4$; $p(c/c)=p(b/b)=1/2$; $p(b/a)=p(c/a)=1/8$; $p(a/b)=p(c/b)=p(a/c)=p(b/c)=1/4$ comprobar que la longitud de Markov para un código compacto $LM = 11/8 < 12/8 = L$
7. Usando el texto del ejercicio 6, Práctico No 3
 - a. Codificar mediante los métodos: Huffman, Fano y Shannon como fuente de Markov de orden 1.
 - b. Relevar para la fuente la estadística de primer orden y realizar una codificación predictiva.
 - c. Realizar una transformación Burrows-Wheller y codificar mediante cuenta de símbolos iguales (RLE).
 - d. Obtener los porcentajes de compresión y comparar los resultados obtenidos.

PRÁCTICO No 4

1. Comprobar para los siguientes conjuntos de probabilidades que $H \leq 1$.
 - a. $p_1 = 0,4$; $p_2 = 0,2$; $p_3 = 0,2$; $p_4 = 0,1$; $p_5 = 0,1$
 - b. $p_1 = 0,4$; $p_2 = 0,4$; $p_3 = 0,1$; $p_4 = 0,1$
 - c. $p_1 = 0,4$; $p_2 = 0,3$; $p_3 = 0,2$; $p_4 = 0,1$
2. Sea una fuente S con eventos s_1, s_2, s_3, s_4 de $p(s_i) = 1/4$ ¿es posible encontrar un $L=1,86$ para cada codificación? ¿Cuál es el mínimo valor que en ese caso puede tomar L ?
3. Repetir el ejemplo anterior pero con $p(s_1)=1/2$; $p(s_2)=1/4$; $p(s_3)=p(s_4)=1/8$
4. Sea la siguiente fuente $S=(s_1, s_2, s_3, s_4, s_5, s_6, s_7)$; $p(s_1)=p(s_2)=1/3$; $p(s_3)=p(s_4)=1/9$; $p(s_5)=p(s_6)=p(s_7)=1/27$. Encontrar un código trinario, para ello encontrar $H(S)$, L y codificar según Huffman.
5. Dada una fuente $S=(s_1, s_2, s_3)$; $p(s_1)=1/2$; $p(s_2)=p(s_3)=1/4$. Codificar por Huffman, Shannon y Fano, encontrar LH , LS y LF .
6. Para el ejemplo anterior codificar por ambos métodos la segunda y tercera extensión viendo la tendencia de la longitud de la fuente original.
7. Dado el siguiente texto: **PABLITOPABLITOPABLOCLAVO**
 - a. Codificar la segunda extensión usando el método de Fano.
 - b. Calcular el porcentaje de compresión y comparar los resultados con los obtenidos codificando Fano como fuente de Markov de orden 1.
 - c. Fundamente la diferencia obtenida en el apartado anterior.
8. Pipeline BWT $\rightarrow$ RLE $\rightarrow$ Huffman: Dada la cadena **TOMATOMATOMAS**
 - a. Aplicar RLE directamente sobre la cadena original

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

- b. Aplicar la transformación Burrows-Wheeler (BWT) y luego RLE sobre la salida de BWT. Comparar resultados

PRÁCTICO DE MÁQUINA 2: Desafío de Proveedores de Compresión

Implementación, descompresión, benchmark y comparación con soluciones de mercado

Escenario del práctico. Cada grupo será considerado un proveedor de software de compresión. El proveedor ofrecerá dos soluciones: una integración de un compresor externo disponible en el mercado (solución tercerizada) y un compresor desarrollado por el propio grupo (solución de autoría propia). Ambas soluciones deberán ser evaluadas sobre los mismos archivos y sus resultados se incorporarán a un benchmark común de la cátedra.

La competencia es técnica y didáctica: no se calificará a un grupo por “ganarle” a otro algoritmo o a una herramienta industrial. Se evaluará la correctitud, la comprensión del método, la calidad de la implementación y la capacidad de interpretar los resultados del benchmark.

Objetivos

- Implementar en **Python** un compresor y un descompresor sin delegar el núcleo del algoritmo propio a una librería de compresión.
- Comprender qué información adicional necesita cada método para permitir la descompresión: cabeceras, diccionarios, árboles, modelos, parámetros o metadatos.
- Demostrar recuperación exacta del archivo original byte a byte.
- Comparar una solución propia, una solución externa asignada y un compresor de referencia común de la cátedra.
- Construir una tabla global de resultados y analizar compromisos entre tamaño, velocidad, complejidad y overhead.

Asignación por sorteo

- Sorteo A — Algoritmo propio: define el método que deberá implementar el grupo.
- Sorteo B — Solución externa: define el compresor de terceros que el grupo integrará y utilizará como alternativa de mercado.

SORTEO A

N.º	Alternativa	Familia	Alcance sugerido
1	LZ + Huffman Estático	Diccionario + estadística	Generar referencias LZ, construir Huffman estático sobre la salida y realizar compresión/descompresión completa.
2	RLE + Shannon	Repeticiones + estadística	Representar rachas mediante RLE y codificar la salida utilizando Shannon. Reconstrucción completa.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

3	BWT + Huffman Estático	Transformación + estadística	Aplicar BWT, codificar su salida mediante Huffman estático e implementar las transformaciones inversas.
4	Huffman dinámico	Adaptativa	Construir y actualizar el árbol durante el procesamiento sin conocer previamente las frecuencias.
5	Shannon Ext. $O(2)$	Estadística / fuente extendida	Trabajar con una extensión de orden 2, calcular probabilidades de pares y generar el código Shannon correspondiente.
6	LZW	Diccionario	Construir el diccionario dinámicamente durante la compresión y reconstruirlo de forma implícita durante la descompresión.
7	Codificación aritmética	Probabilística	Construir el modelo probabilístico, realizar subdivisiones sucesivas del intervalo y recuperar exactamente la secuencia original.
8	Markov orden 1 + Huffman estático	Contexto + estadística	Modelar probabilidades condicionadas por el símbolo anterior y utilizar códigos Huffman asociados a cada contexto.
9	Fano Ext. $O(2)$	Estadística / fuente extendida	Construir una fuente extendida de orden 2 y aplicar Fano sobre los pares obtenidos.
10	PPM-C	Contexto	Implementar PPM-C con modelo de contexto, escapes y descenso de orden durante compresión y descompresión.
11	BWT + RLE + Huffman Estático	Pipeline	Aplicar BWT, representar rachas mediante RLE y codificar estadísticamente la representación resultante con Huffman estático.

Sorteo B — Soluciones externas / de mercado

La expresión “de mercado” incluye tanto software comercial como herramientas abiertas de uso real. El grupo no deberá implementar esta solución: podrá utilizar la aplicación, CLI o herramienta oficial correspondiente, documentando la configuración empleada.

N.º	Solución	Formato / método	Configuración de referencia
1	ZIP / Deflate	ZIP	Preset normal / Deflate. Herramienta ZIP estándar definida por la cátedra.
2	WinRAR	RAR5	Modo normal, sin cifrado ni solid archive salvo indicación de la cátedra.
3	7-Zip	7z / LZMA2	Preset Normal; configuración documentada.
4	PeaZip	PEA / PCOMPRESS2	Formato PEA con compresión equivalente a Deflate nivel 6.
5	WinZip	ZIP / Deflate	Preset normal; sin cifrado.
6	bzip2	BZ2	Configuración estándar fijada por la cátedra.
7	xz	XZ / LZMA2	Nivel 6.
8	Zstandard	ZST	Nivel 3.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

9	Brotli	BR	Quality 5.
10	LZ4	LZ4	Nivel/default rápido documentado.
11	ZPAQ	ZPAQ	Preset definido por la cátedra; sin cifrado.

Compresor base de la cátedra

Para que las métricas comparativas sean consistentes, todos los grupos utilizarán el mismo compresor de referencia: gzip, nivel 6, sin almacenar nombre ni timestamp del archivo.

Configuración de referencia sugerida: gzip -n -6. La referencia no reemplaza a la solución externa sorteada: se utiliza exclusivamente como baseline común para las métricas y, en particular, para el Weissman Score.

Requisitos

- Lenguaje obligatorio para la solución propia: Python.
- Desarrollar dos operaciones claramente separadas: COMPRESOR y DESCOMPRESOR.
- El compresor recibirá un archivo de texto y generará un archivo comprimido propio, por ejemplo con extensión .tdi.
- El descompresor recibirá únicamente el archivo comprimido y deberá reconstruir el archivo original.
- La lectura y escritura se realizará en modo binario cuando sea necesario para preservar exactamente la secuencia de bytes, aunque las entradas de prueba sean archivos de texto.
- No se permite utilizar una librería que implemente directamente el algoritmo propio asignado. Sí se permiten estructuras auxiliares, manejo de bits, archivos, CLI, testing y visualización.
- El archivo comprimido propio deberá contener toda la información mínima necesaria para descomprimirlo sin depender del archivo original.
- El programa deberá detectar errores básicos: archivo inexistente, archivo incompatible, cabecera inválida o datos insuficientes para completar la decodificación.
- Al finalizar deberá informar como mínimo tamaño original, tamaño comprimido, ratio de compresión, porcentaje de ahorro y tiempo de ejecución.
- La validación final deberá demostrar que el archivo reconstruido es idéntico byte a byte al original, para ello será necesario realizar una validación SHA ($\text{SHA256}(\text{original}) = \text{SHA256}(\text{reconstruido})$).

Forma de ejecución sugerida

Interfaz mínima por línea de comandos:

```
python compressor.py entrada.txt salida_grupo.tdi
```

```
python decompressor.py salida_grupo.tdi reconstruido.txt
```

Puede utilizarse otra interfaz siempre que la compresión y descompresión estén claramente separadas y puedan automatizarse para el benchmark.

Salida mínima en pantalla

- archivo de entrada y salida;
- tamaño original y comprimido;
- ratio y ahorro;
- tiempo de compresión o descompresión;
- estado de integridad cuando corresponda;
- parámetros relevantes del algoritmo.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Estructura de proyecto

Elemento	Contenido mínimo
compressor.py	Implementación de la compresión propia.
decompressor.py	Implementación de la descompresión propia.
benchmark.py	Opcional/recomendado: automatiza mediciones y genera resultados reproducibles.
README.md	Algoritmo, instalación, uso, dependencias, formato .tdi, cabecera, solución externa, baseline y ejemplos.
tests/	Archivos de prueba, casos límite y resultados representativos.
results/	CSV/JSON/tablas con resultados del benchmark del grupo.
requirements.txt / pyproject.toml	Solo cuando existan dependencias externas de Python.

Formato comprimido propio y cabecera

Cada grupo definirá su propio formato de archivo. La cabecera deberá ser la mínima necesaria y estar documentada en el README. Puede incluir, según el algoritmo:

- identificador o “magic bytes” del formato y versión;
- tamaño original;
- parámetros del algoritmo (ventanas, orden de contexto, precisión, etc.);
- árbol, tabla de códigos, modelo o información equivalente cuando no pueda reconstruirse implícitamente;
- longitud válida del último byte o cantidad de bits significativos;
- checksum/hash opcional para detectar corrupción.

Benchmark de compresores

Cada grupo evaluará tres soluciones sobre el mismo conjunto de archivos:

1. Compresor propio implementado por el grupo.
2. Solución externa / de mercado asignada por sorteo.
3. Baseline de la cátedra: gzip -6.

Prueba	Tipo	Finalidad
Prueba 1	Archivo muy pequeño	Permite explicar el algoritmo paso a paso y observar el costo de cabecera. No se usa para el ranking temporal.
Prueba 2	Texto natural	Texto de tamaño suficiente (recomendado ≥ 1 MiB) con distribución lingüística real.
Prueba 3	Alta repetición	Texto ≥ 1 MiB con patrones y rachas repetidas.
Prueba 4	Baja repetición	Texto ≥ 1 MiB generado con símbolos aproximadamente equiprobables / baja redundancia.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

La cátedra publicará los archivos exactos del corpus común. Todos los grupos deberán usar esos mismos bytes de entrada para los resultados oficiales.

Métricas a tener en cuenta

Sea S_o el tamaño original en bytes y S_c el tamaño comprimido en bytes.

Ratio de compresión: $R = S_o / S_c$

Interpretación: $R > 1$ indica reducción; cuanto mayor sea R , mejor compresión. Si $R < 1$, el archivo comprimido terminó siendo mayor que el original.

Ahorro de espacio (%): $A = (1 - S_c / S_o) \times 100$

Tamaño relativo (%): $P = (S_c / S_o) \times 100$

Throughput de compresión: $V_c = \text{tamaño original (MB)} / \text{tiempo de compresión (s)}$

Throughput de descompresión: $V_d = \text{tamaño original (MB)} / \text{tiempo de descompresión (s)}$

Overhead de cabecera (%): $O = (H / S_c) \times 100$ (si posee cabecera)

Weissman Score — indicador comparativo

Como métrica complementaria se utilizará el Weissman Score, popularizado por la serie “Silicon Valley” y propuesto para combinar en un solo valor el ratio de compresión y el tiempo, normalizados contra un compresor estándar.

$W = \alpha \times (R / R_{ref}) \times [\log(T_{ref}) / \log(T)]$

Símbolo	Definición
---------	------------

W	Weissman Score de la solución evaluada.
---	---

α	Constante de escala. Para el práctico: $\alpha = 1$.
----------	---

R	Ratio de compresión de la solución evaluada.
---	--

R_{ref}	Ratio de gzip-6 sobre exactamente el mismo corpus.
-----------	--

T	Tiempo de compresión de la solución evaluada.
---	---

T_{ref}	Tiempo de compresión de gzip-6 en el mismo entorno.
-----------	---

Interpretación didáctica: gzip-6 funciona como referencia común. Un valor superior a la referencia indica un mejor compromiso según esta métrica; el resultado debe leerse junto con ratio, tiempos y throughput, nunca de forma aislada. si $W > 1$ entonces el compresor testado es mejor que gzip si el valor es $W < 1$ entonces el valor baseline de gzip es mejor que el compresor desarrollado

Importante sobre Weissman. La fórmula original es sensible a la unidad utilizada para medir el tiempo y requiere un entorno común. Para este práctico los tiempos del Weissman se expresarán obligatoriamente en milisegundos, $\alpha = 1$ y el baseline será siempre gzip-6. Se utilizará como indicador didáctico complementario, no como única medida de calidad.

Para reducir el ruido de archivos individuales, el Weissman oficial puede calcularse sobre el corpus de rendimiento completo:

$R_{global} = (\sum \text{tamaño original}) / (\sum \text{tamaño comprimido})$

$T_{global} = \text{mediana del tiempo total de compresión del corpus}$

Se calculan R_{global} y T_{global} tanto para la solución evaluada como para gzip-6 y luego se aplica la fórmula de Weissman.

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026

Tabla comparativa (ejemplo)

Archivo	Algoritmo	Ratio	Tiempo	Ratio gzip	Tiempo gzip	Weissman
Prueba 1	Huffman	1.20	3 ms	0.95	2 ms	...
Prueba 2	Huffman	1.85	20 ms	2.10	17 ms	...
Prueba 3	Huffman	4.30	18 ms	6.20	15 ms	...
Prueba 4	Huffman	1.01	21 ms	1.00	17 ms	...

Entrega

- URL al repositorio GitHub con el desarrollo completo.
- Código fuente claro, modular y comentado donde sea necesario.
- README con explicación del algoritmo, formato comprimido, instrucciones de ejecución y herramientas externas utilizadas.
- Archivos de prueba y tabla breve con resultados propio vs. externo vs. gzip-6.
- Carga de los resultados obligatorios en la planilla global de la cátedra.
- Demostración en la presentación: comprimir, descomprimir y verificar identidad del archivo.
- Breve análisis de por qué el algoritmo se comportó mejor o peor según el tipo de archivo.

Guión mínimo para la presentación

1. Presentar al grupo como proveedor y explicar las dos soluciones ofrecidas: tercerizada y propia.
2. Explicar brevemente el algoritmo propio y qué metadatos necesita para poder descomprimir.
3. Mostrar una compresión y una descompresión reales.
4. Demostrar integridad byte a byte / SHA-256.
5. Mostrar tabla comparativa propia vs. solución externa vs. gzip-6.
6. Interpretar ratio, ahorro, tiempos, throughput y Weissman Score.
7. Comparar el resultado propio con la tabla global y explicar en qué tipo de datos su algoritmo se desempeña mejor o peor.

FECHA DE ENTREGA: Martes 6 de Octubre

Nota metodológica sobre el Weissman Score

El Weissman Score combina ratio de compresión y tiempo normalizados contra una referencia. Fue creado por Tsachy Weissman y Vinith Misra a pedido de los productores de Silicon Valley. IEEE Spectrum ha señalado tanto su utilidad pedagógica como su sensibilidad a la unidad temporal y a la plataforma de ejecución. Por ese motivo este práctico fija unidad, baseline y protocolo de medición, y conserva métricas separadas de ratio y velocidad.

Referencias sugeridas: IEEE Spectrum, "A Fictional Compression Metric Moves Into the Real World" (2014) y "Weissman Score Calculators Migrate Online, But Metric Needs Some Improvements" (2017).

Teoría de Información

Carrera: Licenciatura en Ciencias de la Computación

Año: 2026